

Overall Merit: Strong accept

Summary Cilk is a C-based runtime for multithreaded parallel programming. A Cilk computation forms a DAG of nonblocking threads, characterized by two quantities: work T_1 (total computation) and critical path T_∞ (infinite-processor time). The scheduler uses randomized work stealing, where idle processors steal the shallowest ready thread from a random victim’s queue.

Empirical results on a CM5 (up to 256 processors) across six applications show near-perfect linear speedup when $T_1/T_\infty \gg P$. A curve fit on the knary benchmark yields $T_P \approx T_1/P + 1.5 \cdot T_\infty$ with $R^2 = 0.989$. For “fully strict” programs, the authors prove optimal bounds: space $S_P \leq S_1 P$, expected time $O(T_1/P + T_\infty)$, and expected communication $O(T_\infty P S_{\max})$.

Strengths

1. The paper delivers on both theory and practice. The provably optimal bounds are matched by tight empirical fits ($R^2 > 0.98$ on knary and $R^2 = 0.9994$ on *Socrates). Programmers can actually use T_1 and T_∞ to reason about performance without thinking about scheduling.
2. The space bound $S_P \leq S_1 P$ is a strong practical result. *Socrates allocates at most 405 closures per processor even when executing 51 million threads on 256 processors. The busy-leaves proof is clean and easy to follow.
3. Stealing from the shallowest level is well-motivated by two independent arguments: shallow threads contain more work (fewer steals needed), and they sit on the critical path (ensuring progress). Table 4 confirms this: fib executes 17 million threads but migrates only 6170 on 256 processors.

Weaknesses

1. The theoretical guarantees only hold for fully strict programs, and all benchmarks happen to be fully strict. It is unclear how common non-strict programs are or how badly the scheduler degrades for them.
2. The explicit continuation-passing style is a real usability cost. Manual `send_argument` calls are verbose and error-prone compared to fork-join. Later Cilk versions moved to `spawn/sync` precisely because this was a barrier to adoption.

Detailed Comments The *Socrates anecdote in Section 5 is the best part. The authors found an “improvement” that sped up the program on 32 processors, but measuring T_1 and T_∞ separately revealed it traded less work for a longer critical path. Their model predicted it would be slower on

the 512-processor tournament machine, which they later confirmed. This is a concrete payoff of the work/critical-path abstraction.

I also appreciate the honesty about overhead: a spawn costs roughly $8\text{--}9\times$ a C function call (from fib's efficiency of 0.116), and the paper states this directly rather than hiding behind benchmarks with long threads.

Discussion Questions

1. The stealing policy ignores data locality. Modern architectures have deep cache hierarchies and NUMA effects. How would you modify the policy to account for locality, and what would you give up in the theoretical bounds?
2. The fully strict restriction means threads send results only to their parent's successor. Are most real parallel programs effectively fully strict, or does this rule out common patterns like futures or arbitrary task graphs?